Reinforcement Learning for the *Codenames* Spymaster Task

Authors
Pranshul Bhatnagar, Aesha Gandhi, Gaurav Law, Jay Wu

Introduction

Our project studies the spymaster role in *Codenames* as a reinforcement learning problem. In the game, the spymaster sees the hidden roles of all words on the board and must give a one-word clue and a number that help a teammate guess friendly words while avoiding opponent words, neutral words, and especially the assassin. This is a good RL setting because each action requires balancing semantic similarity, long-horizon planning, and risk. A clue that is too broad may accidentally point to a dangerous word, while a clue that is too narrow may fail to make enough progress toward winning.

The main goal of the project was to build a learning-based spymaster that uses the semantic structure of the board rather than relying only on hand-designed rules. Following the original proposal, we focused on four ideas: reward shaping, behavioral cloning, Soft Actor-Critic (SAC), and Hindsight Experience Replay (HER). Our final project includes all four. We built a goal-conditioned version of the task, a strong greedy semantic baseline, demonstration generation for behavioral cloning, and an SAC training pipeline with optional HER and ablations.

More broadly, this project sits at an interesting boundary between reinforcement learning and language-like reasoning. *Codenames* is not a standard control task in which the agent chooses from a small set of physical actions. Instead, the spymaster must choose a clue that is semantically related to some words on the board and semantically separated from others. That makes the problem a useful testbed for whether RL methods can be adapted to settings where representation quality matters as much as exploration and optimization.

Our original hypothesis was that a purely model-free RL approach would struggle unless we injected more structure into the problem. In particular, we expected that three additions would matter: a semantic representation of clues and board words, a shaped reward that gives useful turn-level feedback, and a demonstration-based warm start so the agent does not begin from completely random behavior. The final project was designed to test that hypothesis.

Problem Statement

The task is to learn a policy for the spymaster in *Codenames*. At each turn, the spymaster sees the full board with hidden roles and must output a one-word clue and a positive integer count. A teammate then uses that clue to guess board words. The goal is to reveal all friendly words before hitting the assassin, while avoiding opponent and neutral words whenever possible.

From an RL perspective, this creates several challenges. First, the action space is unusual because clues are words, not ordinary low-dimensional controls. Second, the reward is sparse at the game level: the agent may make several reasonable clueing decisions before eventually losing. Third, clue quality depends on semantic relationships, so the representation of words matters directly to the learning problem. Finally, there is a tradeoff between ambition and safety. A clue that aims at many friendly words can shorten the game, but it also increases the chance of pointing the guesser toward a bad word.

We therefore formulated the problem as a semantic decision task. Instead of trying to model every aspect of open-ended language generation, we treated clueing as selecting a point in a semantic space and then decoding that point into a legal clue. This preserved the spirit of the game while making the problem compatible with the RL tools we wanted to use.

Project Overview

We modeled the *Codenames* spymaster as an agent that observes the full hidden board and outputs a clue. The environment then simulates a fixed teammate guesser, evaluates the outcome of that clue, and returns reward to the spymaster.

We started with a 5x5 board, using a fixed 400-word *Codenames* vocabulary in `data/raw/codenames_words.txt`. In the default setup, the board contains 8 friendly words, 8 opponent words, 8 neutral words, and 1 assassin. The observation includes semantic embeddings for the board words, a pairwise board similarity matrix, role labels, a mask for unrevealed words, and a goal representation. The environment is goal-conditioned: on each turn it chooses a small subset of remaining friendly words as the current target, and the agent is rewarded for giving a clue that helps reveal that subset safely.

The agent's action is represented as a continuous vector. Most of the vector corresponds to a clue embedding, and the final scalar corresponds to the clue count. The environment decodes that vector into the nearest legal clue word from a filtered clue vocabulary and converts the count into an integer between 1 and 9.

The final implementation uses sentence-transformer embeddings from `all-MiniLM-L6-v2`. The clue vocabulary is drawn from a curated list, filtered to single-word clues, and screened to remove obvious illegal overlaps with board words. In the base configuration, the clue vocabulary is capped at 8000 candidate words.

This design means that the learned agent is not generating language in the full open-ended sense. Instead, it is solving a structured semantic control problem. For our purposes, that was a feature rather than a limitation: it let us focus on the central RL question of whether the agent could learn to give better clues over time given the board structure, reward function, and demonstrations.

Approach

Our approach combines a strong heuristic baseline with off-policy reinforcement learning. The baseline searches over legal clue words and chooses the clue and count that maximize a cosine-margin objective: the clue should be close to the intended friendly targets and far from the most dangerous alternative. This baseline serves two purposes. First, it is the main comparison point for the learning-based agent. Second, it generates demonstration trajectories for behavioral cloning.

Behavioral cloning adds a supervised learning step to our unsupervised problem, in which we train a policy to imitate actions from an expert or heuristic policy. In our project, the "expert" is the greedy semantic baseline. Before any RL fine-tuning, we optionally pretrain the actor on these demonstration trajectories so that it starts from a policy that already gives plausible clues rather than from random behavior. Concretely, the BC stage minimizes mean-squared error between the actor's output action vector and the demonstration action vector. We also optionally seed the replay buffer with those demonstration transitions, which gives the RL algorithm access to more informative early experience.

In the final version of the project, we strengthened BC in two ways. First, we effectively augment each demonstration board many times by fixing a base board vocabulary but shuffling the board configuration during training, which exposes the policy to many semantically related board-role layouts instead of a single static arrangement. Second, we add an auxiliary cosine-margin loss during BC pretraining. This extra loss encourages the policy not only to imitate the demonstration action, but also to produce clues whose semantic margin is high under the same cosine-margin idea used by the reward function.

Our main reinforcement learning algorithm is Soft Actor-Critic (SAC). SAC is an off-policy actor-critic method for continuous action spaces. Very roughly, it learns a policy that tries to maximize reward while also maintaining enough randomness to keep exploring, which is where the "soft" part of the name comes from. We chose SAC because our action is a continuous vector rather than a small discrete set of clues. In our project, the SAC actor outputs a clue embedding and a clue count, and the environment decodes that output into a legal game action.

We combine SAC with Hindsight Experience Replay (HER). HER is a replay strategy designed for goal-conditioned problems, especially when reward is sparse. The basic idea is that even if an episode does not achieve its original goal, it may still have achieved some other useful goal, and the algorithm can learn from that relabeled experience. This is well-matched to our setting because each turn is framed around a desired subset of friendly words. A clue may fail to reveal the exact intended subset but still make meaningful partial progress. By relabeling experience in terms of achieved goals, HER helps SAC learn from more of the trajectories it collects instead of treating most failures as equally uninformative.

Reward shaping was another central part of the approach. Reward shaping means augmenting the learning signal with intermediate feedback rather than relying only on the final success or failure of the episode. In our project, the reward combines three signals. First, we use a semantic clue-margin term that measures how well the clue separates intended friendly words from dangerous words. Second, we add penalties when the guesser reveals an opponent, neutral, or assassin word. Third, we add a turn penalty when the desired goal subset is not achieved. The purpose of shaping was to avoid a purely sparse "win or lose" signal, which would make the task much harder to learn from.

Putting these pieces together, our method works in stages. The semantic baseline gives us both a non-learning comparator and a source of demonstrations. Behavioral cloning uses those demonstrations to initialize the policy. SAC then fine-tunes that policy through interaction with the environment and HER makes the replay process more sample-efficient by learning from partial-goal outcomes. Reward shaping gives the agent denser turn-level feedback throughout training.

We considered more standard on-policy baselines such as vanilla PPO and A2C, including approaches emphasized in prior work on game-playing RL. However, we did not choose them as our primary method because our final action space is continuous and semantically structured, and our project also depends heavily on replaying demonstration data and relabeled goal-conditioned experience. SAC fit that setup better than PPO or A2C: it is designed for continuous actions, it is off-policy, and it works naturally with replay buffers and HER. By contrast, vanilla PPO and A2C would have made it harder to use demonstrations and hindsight replay as centrally as we wanted, while also being less naturally aligned with our continuous clue-embedding formulation.

Method Summary Table

Component	**Final choice**	**Why we used it**
Task formulation	Goal-conditioned spymaster-only RL	Simplifies evaluation and isolates clue quality
Board setup	5x5 *Codenames* board	Matches the standard game format for the project
Representation	Sentence-transformer embeddings	Gives a usable semantic space for both clues and board words
Action space	Continuous clue embedding + scalar count	Makes SAC applicable while preserving semantic flexibility
Guesser	Fixed greedy teammate	Stabilizes evaluation and reduces multi-agent complexity
Baseline	Greedy cosine-margin clue generator	Strong semantic baseline and source of demonstrations
RL algorithm	SAC	Off-policy continuous-control method and well suited to the action design
Replay strategy	HER	Helps with partial progress and sparse-goal structure

Component	Final choice	Why we used it
Pretraining	Behavioral cloning from greedy demonstrations, with board augmentation and auxiliary cosine-margin loss	Improves initialization and encourages BC to generalize beyond memorized boards
Reward	Margin shaping + bad-guess penalties + goal penalty	Gives denser signal than pure win/loss reward

Related Ideas That Influenced the Project

A few ideas heavily influenced the final design. SAC and HER directly motivated the RL side of the project, since we wanted an off-policy algorithm that works well with continuous actions and a replay strategy that can help on sparse-goal problems. Sentence-BERT style embeddings influenced the semantic representation, since the core difficulty in *Codenames* is measuring whether a clue groups the right words and separates them from the wrong ones. More broadly, the project was influenced by prior work on embedding-based semantic reasoning in word games and language-adjacent control tasks.

The most important influence was not a single prior result, but a pattern from recent ML work: when the task has language-like structure, raw RL is often not enough on its own. Representation learning, shaping, imitation, and task reformulation often matter just as much as the choice of optimizer. Our project followed that pattern closely.

Baselines and Experimental Setup

Our baseline is the greedy spymaster described above; this is a meaningful baseline because it already uses semantic information and is much stronger than random clueing. Our final rollout-debug comparison uses four methods: the greedy baseline; a pure BC pretraining pipeline; BC + SAC + HER without reward shaping; and BC + SAC + HER with reward shaping. These comparisons test the central hypothesis from the proposal: semantic shaping and imitation-based warm starts should improve learning over plain RL, and that the strengthened BC stage can provide a better initialization than imitation alone.

The current reproducible training setup uses a 5x5 board, 8 friendly words, a goal size of 3, and a rollout-debug configuration based on `training_pipeline_5x5.yaml` with common notebook overrides. In the notebook reported here, the clue vocabulary cap is reduced to 1200, pipelines with behavioral cloning use 6 demonstration episodes and 2 BC epochs, and SAC training uses 120 total timesteps with smaller network settings for quick comparison. HER uses 4 sampled goals with the future selection strategy. Our evaluation reports mean return, mean turns, win rate, assassin rate, and friendly reveal rate. Those metrics were chosen because win rate alone is not enough in this domain: an agent can be conservative and avoid the assassin while revealing too few friendly words, or it can reveal many friendly words but still fail through risky clues.

To reproduce the current pipeline, one can create a Python environment, install the project requirements, run the main SAC/HER training configuration, and then run the ablation suite using the same base settings. We also built an end-to-end smoke test to verify the full pipeline before larger experiments.

Experimental Settings Table

Setting	Value
Board size	5 x 5
Friendly / Opponent / Neutral / Assassin	8 / 8 / 8 / 1
Max turns	20
Max clue count	9
Goal size	3

Setting	Value
Embedding model	`all-MiniLM-L6-v2`
Clue vocabulary size cap in rollout debug	1200
BC enabled	Yes
Demonstration episodes in rollout debug	6 for pipelines with BC enabled
BC epochs in rollout debug	2 for pipelines with BC enabled
BC auxiliary cosine-margin loss weight	0.5 for the BC-heavy pipeline
RL algorithm	SAC
HER enabled	Yes
Total timesteps in rollout debug	120
Evaluation episodes in rollout debug	2
Seed in current base config	43

Baseline and Ablation Table

Method	Reward shaping	Behavioral cloning	HER	Notes
Greedy	No	No	No	Non-learning semantic heuristic baseline
BC	No	Yes	No	Behavioral cloning pretraining only, using augmented demonstrations and auxiliary cosine-margin loss
BC + SAC + HER	No	Yes	Yes	Tests whether RL fine-tuning with HER improves on BC without shaped reward
BC + SAC + HER + Reward Shaping	Yes	Yes	Yes	Full method with shaped reward enabled

Results

Our finalized comparison evaluates the four main pipelines under a common rollout setting. We report both evaluation metrics and a single traced rollout summary. The evaluation metrics capture aggregate performance across evaluation episodes, while the rollout summary helps illustrate how each trained policy behaves in a concrete game trajectory.

The main pattern in these results is that the greedy baseline remains the strongest method in this small-scale debug setting: it is the only pipeline with nonzero evaluation win rate, and it also achieves the best traced rollout reward. The BC-only pipeline performs the worst in the traced rollout, hitting the assassin and ending with the lowest total reward. Both SAC-based pipelines perform better than BC-only in the traced rollout and complete the trace without hitting the assassin, but neither improves evaluation win rate beyond 0.0. Reward shaping produces behavior that is extremely similar to unshaped BC + SAC + HER, with slightly worse traced total reward in this particular notebook run. Overall, the 5x5 rollout-debug notebook

suggests that the full pipeline is behaving more stably than BC-only, but the setting is still too small to show a clear evaluation advantage over the greedy baseline.

Main Results Table

Method	Reward shaped weight	Demo transi-tions	Eval win rate	Eval assassin rate	Trace steps	Trace total reward	Trace result
Greedy	0.0	0	0.5	0.5	9	-8.000	Won, no assassin
BC	0.0	58	0.0	1.0	22	-88.000	Lost, assassin hit
BC + SAC + HER	0.0	58	0.0	0.5	20	-44.500	Won, no assassin
BC + SAC + HER + Reward Shaping	1.0	58	0.0	0.5	20	-45.486	Won, no assassin

Evaluation Interpretation Table

Method	What it tests	Observed behavior in rollout debug	Takeaway
Greedy	Upper-bound heuristic baseline	Best evaluation win rate and best traced rollout reward overall	Strong semantic heuristic remains the clearest winner in the 5x5 debug notebook
BC	Value of imitation alone	Evaluation win rate stays at 0.0 and the traced rollout hits the assassin with the worst total reward	BC alone is not sufficient in this setting, even with stronger pretraining
BC + SAC + HER	RL fine-tuning without shaped reward	Evaluation remains weak, but the traced rollout succeeds without assassin and is much stronger than BC-only	RL fine-tuning improves traced behavior relative to BC-only in this notebook run
BC + SAC + HER + Reward Shaping	Full method with shaped reward	Very similar to unshaped SAC + HER, but with slightly lower traced reward in this run	Reward shaping does not show a clear advantage at this small debug scale

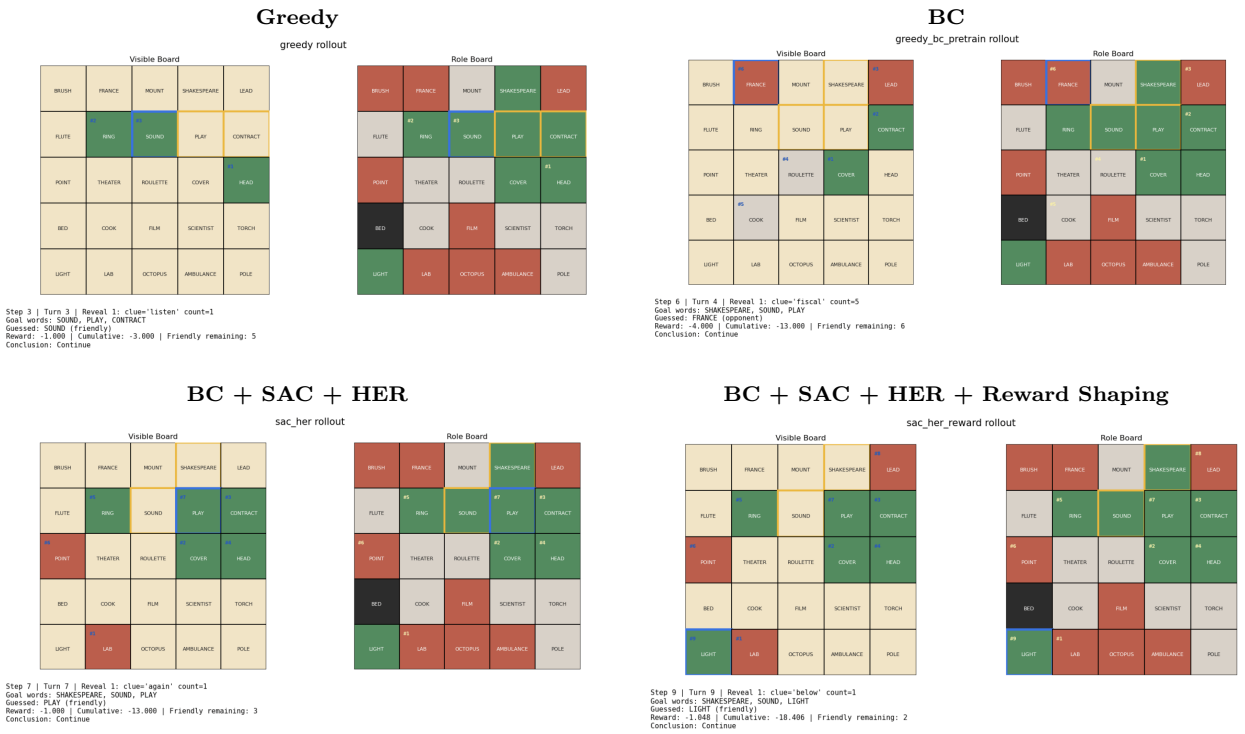

Figure 1: Side-by-side rollout snapshots for the four evaluated pipelines. These figures provide a qualitative comparison of how each method organizes clueing behavior on the same style of 5x5 board.

Result Discussion

The final comparison suggests three main takeaways. First, the greedy baseline remains the strongest method overall in this 5x5 debug benchmark, since it is the only pipeline with nonzero evaluation win rate and also has the best traced rollout reward. Second, behavioral cloning by itself is not enough: despite stronger pretraining, augmentation, and the auxiliary cosine-margin loss, the BC-only pipeline performs worst among the learned methods in the traced rollout and still has zero evaluation win rate. Third, adding SAC + HER improves traced behavior relative to BC-only, but in this notebook run reward shaping does not produce a clear advantage over the unshaped SAC + HER variant.

Taken together, these results suggest that the project is successfully learning something about clueing behavior beyond pure imitation, but the small rollout-debug regime is still dominated by the handcrafted greedy heuristic. The final comparison therefore supports a cautious conclusion: the full RL pipeline is working and is meaningfully different from BC-only, but it does not yet outperform the greedy baseline in this compact benchmark.

Held-Out Board Generalization

In addition to the in-distribution 5x5 rollout-debug comparison, we ran a separate held-out board generalization test, training only the BC + SAC + HER + Reward Shaping pipeline once, then evaluates that trained agent on five new 5x5 boards built from words that do not appear in the final generalization. This gives a cleaner check of whether the policy transfers beyond the board distribution it saw during the debug comparison.

Training reference	Win rate	Assassin rate	Mean return	Friendly reveal rate
BC + SAC + HER + Reward Shaping (train-side evaluation)	0.0	1.0	-68.682	0.407

Board	Status	Assassin hit	Turns	Reward	Friendly revealed	Friendly remaining
Board 1	Fail	Yes	5	-54.499	0	9
Board 2	Fail	Yes	5	-49.788	2	7
Board 3	Fail	Yes	8	-58.950	3	6
Board 4	Fail	No	20	-50.004	8	1
Board 5	Fail	Yes	5	-49.755	2	7

Held-out summary	Boards tested	Win rate	Assassin rate	Mean reward	Mean friendly revealed
Generalization evaluation	5	0.0	0.8	-52.599	3.0

These results show that the learned policy does not generalize well to new boards. It failed on all five held-out boards, and in four of the five cases the run ended by hitting the assassin very early. The one exception, Board 4, is also informative: the model avoided the assassin and revealed 8 friendly words, but it still failed by ending the episode with one friendly word left unrevealed after 20 turns. This suggests that the model's failure on new boards is not only a safety problem; it is also a clue-quality and efficiency problem. On unseen word sets, the policy seems unable to consistently produce clues that either safely separate good words from bad ones or finish the board quickly enough to win.

Deviations from the Proposal

The final project stayed close to the proposal in spirit, but there were several important deviations. First, we fixed the teammate guesser instead of trying to learn both players jointly. We made this decision because jointly training a spymaster and a guesser would have introduced much more instability and would have made it harder to tell whether gains came from better clueing or simply from a different teammate policy. Fixing the guesser turned the project into a much cleaner controlled experiment.

Second, we did not treat clue generation as unconstrained language generation. Instead, the policy outputs a continuous embedding that is decoded to the nearest legal clue in a fixed vocabulary. This was mainly a practical decision. It keeps the action space manageable for SAC, preserves valid game actions, and still allows the learned policy to operate in a semantic space rather than a small hand-coded action set.

Third, the final system consolidated around sentence-transformer embeddings rather than experimenting with a larger variety of symbolic or lexical resources. This helped us keep the project coherent and reproducible. Once the environment, baseline, and reward function were all using the same semantic representation, the rest of the pipeline became easier to build and debug.

Finally, the current training scale is still modest. Much of the project effort went into getting a correct, inspectable, reproducible pipeline rather than immediately scaling to long expensive runs. As a result, the final comparison in this report should be interpreted as a completed small-scale 5x5 benchmark rather than a large-budget final training study.

Taken together, these deviations reflect a general theme of the project: narrowing the scope was often necessary to make the core RL question answerable. Instead of asking whether we could solve all of *Codenames* at once, we asked whether a spymaster can learn better clueing behavior in a controlled semantic environment. That narrower question was much more realistic for the class project timeline and still scientifically

meaningful.

Key Takeaways

One of the main things we learned is that this problem is much more tractable once it is expressed in semantic space. A clue in *Codenames* is not naturally a low-dimensional symbolic action. Thinking of clueing as movement in an embedding space made several parts of the project line up at once: the action space, the baseline, the reward shaping, and the guesser model all became easier to define.

We also learned that sparse reward alone would likely be too weak for this task. If the only signal is eventual success or failure, then most early trajectories look equally bad. Reward shaping gave the agent feedback about whether a clue was at least semantically promising, even if the full turn or full game was not yet successful. That did not solve the problem by itself, but it made the learning problem much better structured.

Another lesson was that the greedy baseline was useful for more than evaluation. It became an important tool for debugging and for generating demonstrations. In a task like this, random exploration is unlikely to stumble into good semantic clues often enough to be efficient. Behavioral cloning gives the policy a much better starting point, and even when the final learned policy does not beat the baseline, the baseline still helps us build and understand the system.

More broadly, the experience reinforced the fact that combining methods is often harder, but also more realistic, than using any one method in isolation. Our project was not "just SAC" or "just HER." The working version depended on a combination of representation design, reward shaping, imitation, and off-policy RL. That layering was one of the most important practical takeaways from the project.

There was also a more practical lesson about experimentation. In a project like this, it is easy to confuse "the method is weak" with "the pipeline is broken." Building small smoke tests and debugging runs first turned out to be essential. That process did not make the final learned policy stronger by itself, but it made our conclusions more trustworthy because we could separate infrastructure issues from actual learning difficulty.

The held-out board results added another important lesson: success on a small fixed training distribution does not automatically translate into general semantic reasoning on new boards. Our model could produce plausible behavior in the controlled 5x5 pipeline, but it failed consistently on unseen boards, usually by hitting the assassin early and occasionally by making some progress without finishing. That suggests the current method is learning board-specific regularities more than a robust transferable clueing strategy, which makes cleaner generalization-focused evaluation an important part of future work.

Project Evolution

The path to the current system was iterative rather than linear. Early on, the main question was how to represent clues and board words in a way that made the game learnable. Once we committed to embedding-based semantics, it became natural to build a cosine-margin baseline, and that baseline quickly became the center of the rest of the system. It produced clues that were good enough to use as demonstrations, and it also exposed when the clue vocabulary or similarity structure was not behaving as expected.

A second important step was separating correctness from performance. We built smoke-test configs and notebooks that verify environment reset and step logic, clue decoding, demonstration generation, and short SAC/HER runs. Those small tests were essential because they let us debug the mechanics of the system without confusing infrastructure bugs with genuine learning difficulty. In hindsight, that separation saved a lot of time.

A third lesson from the development process was that simplifying the game can make the scientific story stronger. Fixing the guesser, constraining clues to a filtered vocabulary, and using coherent target subsets all reduce realism somewhat, but they also make the problem better defined and easier to evaluate. Those

simplifications were not accidental shortcuts; they were part of turning a very open-ended game into a tractable RL problem.

If we were continuing the project beyond the class, the next stage would be to scale experiments rather than redesign the method from scratch. At this point the project already has a clear environment, a meaningful baseline, and a structured RL pipeline. The remaining work is mostly empirical: run longer training, evaluate across seeds, and determine which combination of shaping, imitation, and HER actually matters most.

Limitations and Future Work

The current project still has several clear limitations. The guesser is fixed rather than learned, clue generation is limited to a candidate vocabulary, and the current training budgets are still relatively small. The environment also approximates the real game's legal clue rules rather than modeling every edge case. Because of those choices, the project should be understood as a structured RL study of the spymaster role, not as a complete superhuman *Codenames* system.

The most obvious next steps are to scale up training, evaluate across multiple random seeds, test more embedding models, and complete the ablation study with stronger final runs. A particularly important next step would be to redesign the experimental split so that generalization is measured more cleanly. In the current rollout-debug setting, training and evaluation reuse a fixed board-word set with shuffled configurations, which is useful for debugging but does not cleanly test transfer to new boards. A stronger design would be to pre-generate a board dataset, split it into train, validation, and test boards, apply augmentation only to the training boards, use BC and SAC/HER only on the training split, tune hyperparameters on validation boards, and reserve the held-out test boards for final evaluation. That would make it possible to report not only performance on the training distribution, but also whether the policy generalizes to unseen board configurations.

Another promising direction is to rethink the similarity metric itself. Our current setup relies heavily on cosine similarity in a static embedding space, which is simple and effective but also limited. Cosine similarity does not explicitly model semantic sparsity, polysemy, or sense ambiguity, all of which matter in a game like *Codenames* where one clue can evoke several meanings at once. A natural next step would be to explore alternatives that better capture sparse semantic structure or sense-specific meaning, such as contextual or sense-disambiguated embeddings, or similarity measures that weight only the most relevant semantic dimensions instead of treating every dimension equally. This could be especially important for clues whose correct interpretation depends on selecting one sense of a word while suppressing another.

Beyond that, a natural extension would be to compare the current continuous embedding-action approach against a discrete-action formulation over clue vocabularies, or to replace the fixed guesser with stronger or more human-like teammate policies.

Reproducibility Checklist

Item	Included in project?	Notes
Fixed board vocabulary	Yes	Same board word source across runs
Fixed clue vocabulary construction	Yes	Filtered and capped clue set
Configurable hyperparameters	Yes	Main settings stored in config files
Deterministic seeds	Yes	Seeded runs supported
Baseline implementation	Yes	Greedy semantic baseline available
Ablation setup	Yes	Core variants already defined
Smoke-test pipeline	Yes	End-to-end debugging workflow included

Item	Included in project?	Notes
Final rollout-debug comparison results	Yes	Included in the report tables from the executed notebook

Conclusion

This project implemented a full RL pipeline for the *Codenames* spymaster task, grounded in the ideas from our proposal. The final system includes a custom goal-conditioned environment, semantic embeddings for board words and clues, a greedy semantic baseline, behavioral cloning from demonstrations, and SAC training with optional HER and ablations. This now includes executed rollout-debug comparison results across the main pipelines, and the project provides a reproducible framework for studying semantic decision-making in a word-based game.

The biggest conceptual contribution of the project is not any single algorithmic component. It is the way the task was reformulated so that reinforcement learning becomes plausible at all. By treating clueing as semantic control in embedding space, and by combining reward shaping with imitation and HER, we turned a difficult language-like problem into something that can be explored with RL tools.

Project Repository

The project codebase, configurations, notebooks, and supporting materials are available at:

https://github.com/pb0104/Codenames-Spymaster-RL/tree/main

Documentation of Sources Used for Code, Including LLMs

The implementation relies on external libraries rather than copied standalone code. In particular, Stable-Baselines3 was used for SAC and HER support, Gymnasium for the environment interface, PyTorch for supervised actor pretraining, and Sentence-Transformers for semantic embeddings. The project also used general library documentation for those packages during development.

We used generative AI tools to help guide us formulate the code structure for this project, particularly the environment and game setup.

Bibliography

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." *Proceedings of ICML*, 2018.

Andrychowicz, M., Wolski, F., Ray, A., et al. "Hindsight Experience Replay." *Proceedings of NeurIPS*, 2017.

Reimers, N., and Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of EMNLP-IJCNLP*, 2019.

Stable-Baselines3 documentation and source code.

Gymnasium documentation.

PyTorch documentation.

Sentence-Transformers documentation.

Chvatil, Vlaada. *Codenames*. Czech Games Edition, 2015.